```
using System;
using System.Collections.Generic;
using System.Linq.Expressions;
using System.Text;

namespace cstorchcl.compiler
{
    public class LocalExpressionVisitor : ExpressionVisitor
    {
        private readonly StringBuilder _bodyCode = new StringBuilder();

        // Отслеживаем только используемые тензоры для сигнатуры аргументов ядра
        public Dictionary<string, LocalVariable> UsedTensors { get; } = new
Dictionary<string, LocalVariable>();

        /// <summary>
        /// Генерирует оптимизированный исходный код OpenCL ядра с
подставленными константами и функциями дропаута.
        /// </summary>
        public string GenerateKernelSource()
        {
            var source = new StringBuilder();

            // Встраиваем ваш быстрый генератор случайных чисел и логику
дропаута в начало кода
            source.Append(@"// Встроенные утилиты генерации шума и Dropout
inline float nn_rand(int tid, unsigned int seed) {
    unsigned int x = tid + seed;
    x = (x ^ 61) ^ (x >> 16);
    x += (x << 3);
    x = x ^ (x >> 4);
    x *= 0x27d4eb2d;
    x = x ^ (x >> 15);
    return (float)x / 4294967296.0f;
}

inline float nn_dropout(float x, float p, int tid, unsigned int seed) {
    if (p <= 0.0f) return x;
    if (p >= 1.0f) return 0.0f;
    float r = nn_rand(tid, seed);
    if (r < p) {
        return 0.0f;
    } else {
        return x / (1.0f - p);
    }
}

");

            // Формируем сигнатуру ядра. Теперь здесь ТОЛЬКО буферы и их длины!
            source.Append("kernel void fused_elementwise_kernel(\n");

            int tensorIdx = 0;
            foreach (var kvp in UsedTensors)
            {
                source.Append($"    global float* {kvp.Key}, const int
{kvp.Key}_len");
                if (tensorIdx < UsedTensors.Count - 1) source.Append(",\n");
                tensorIdx++;
            }

            source.Append(") {\n");
            source.Append("    int tid = get_global_id(0);\n\n");
```

```
            // Оптимизация private-памяти (регистров GPU)
            source.Append("    // Чтение во внутренние регистры\n");
            foreach (var kvp in UsedTensors)
            {
                source.Append($"    float priv_{kvp.Key} = {kvp.Key}[min(tid,
{kvp.Key}_len - 1)];\n");
            }
            source.Append("\n");

            // Тело вычислений
            source.Append("    // Вычисления над регистрами\n");
            source.Append(_bodyCode.ToString());
            source.Append("\n");

            // Безопасное сохранение результатов во VRAM
            source.Append("    // Сохранение результатов во VRAM\n");
            foreach (var kvp in UsedTensors)
            {
                source.Append($"    if (tid < {kvp.Key}_len) {{ {kvp.Key}[tid] =
priv_{kvp.Key}; }}\n");
            }
            source.Append("}");

            return source.ToString();
        }



        protected override Expression VisitMethodCall(MethodCallExpression node)
        {
            // 1. Обработка метода .Assign(...) -> замена знака равенства '='
            if (node.Method.Name == "Assign" && node.Object is MemberExpression
leftVariable)
            {
                string varName = leftVariable.Member.Name;
                _bodyCode.Append($"    priv_{varName} = ");

                // Разбираем то, что передано внутрь Assign(...)
                Visit(node.Arguments[0]);
                _bodyCode.Append(";\n");
                return node;
            }

            // 2. Обработка стандартных математических функций активации
            if (node.Method.DeclaringType == typeof(nnclloc))
            {
                string funcName = node.Method.Name.ToLower();

                if (funcName == "dropout")
                {
                    _bodyCode.Append("nn_dropout(");
                    if (node.Arguments[0] is MemberExpression varMember)
                    {
                        _bodyCode.Append($"priv_{varMember.Member.Name}, ");
                    }
                    var pLambda =
Expression.Lambda<Func<float>>(node.Arguments[1]);
                    float pVal = pLambda.Compile()();

_bodyCode.Append(pVal.ToString(System.Globalization.CultureInfo.InvariantCulture
) + "f, tid, ");

                    var seedLambda =
Expression.Lambda<Func<uint>>(node.Arguments[2]);
```

```
                    uint seedVal = seedLambda.Compile()();
                    _bodyCode.Append(seedVal + "u)");
                }
                else
                {
                    _bodyCode.Append($"{funcName}(");
                    if (node.Arguments[0] is MemberExpression argMember)
                    {
                        _bodyCode.Append($"priv_{argMember.Member.Name}");
                    }
                    _bodyCode.Append(")");
                }
                return node;
            }

            return base.VisitMethodCall(node);
        }

        // Переписываем VisitBinary для обработки простых операций внутри Assign
(например, A / q)
        protected override Expression VisitBinary(BinaryExpression node)
        {
            Visit(node.Left);

            string opStr = node.NodeType switch
            {
                ExpressionType.Add => " + ",
                ExpressionType.Subtract => " - ",
                ExpressionType.Multiply => " * ",
                ExpressionType.Divide => " / ",
                _ => throw new NotSupportedException($"Операция {node.NodeType}
не поддерживается в JIT.")
            };

            _bodyCode.Append(opStr);
            Visit(node.Right);

            return node;
        }

        // Обновляем VisitMember, чтобы он правильно подставлял имена регистров
в правых частях уравнений
        protected override Expression VisitMember(MemberExpression node)
        {
            if (node.Type == typeof(LocalVariable))
            {
                string name = node.Member.Name;
                if (!UsedTensors.ContainsKey(name))
                {
                    var objectMember = Expression.Convert(node, typeof(object));
                    var getterLambda =
Expression.Lambda<Func<object>>(objectMember);
                    var localBuffer = (LocalVariable)getterLambda.Compile()();
                    UsedTensors.Add(name, localBuffer);
                }

                // Если мы уже внутри парсинга выражения, то подставляем имя
приватного регистра GPU
                _bodyCode.Append($"priv_{name}");
                return node;
            }

            // Если это внешняя переменная типа float q
            if (node.Type == typeof(float) || node.Type == typeof(double))
```

```
            {
                var objectMember = Expression.Convert(node, typeof(object));
                var getterLambda =
Expression.Lambda<Func<object>>(objectMember);
                float val = Convert.ToSingle(getterLambda.Compile()());


_bodyCode.Append(val.ToString(System.Globalization.CultureInfo.InvariantCulture)
+ "f");
                return node;
            }

            return base.VisitMember(node);
        }

    }
}
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.CSharp.Syntax;

namespace cstorchcl.compiler
{
   public class ActivationConfig
   {
      public string ClForwardFunc { get; set; }
      // Обрати внимание: для некоторых функций (как ReLU) производная считается по
ВХОДУ priv_x,
      // а для некоторых (как Sigmoid) — эффективнее по ВЫХОДУ priv_y. Сделаем по
умолчанию по ВХОДУ.
      public string ClBackwardFormula { get; set; }
   }
   public class LocalRoslynTranspiler : CSharpSyntaxRewriter
   {
      private readonly StringBuilder _bodyCode = new StringBuilder();
      private readonly StringBuilder _backwardBodyCode = new StringBuilder();

      public HashSet<string> DiscoveredTensors { get; } = new
HashSet<string>(StringComparer.OrdinalIgnoreCase);
      private Dictionary<string, float> _externalConstants;

      // Словарь для быстрого доступа к метаданным тензоров во время обхода AST
      private Dictionary<string, core.Tensor> _tensorMeta;
      private static readonly Dictionary<string, ActivationConfig> Activations =
new(StringComparer.OrdinalIgnoreCase)
{

   // Теперь везде строго {0} - это имя приватного регистра ВХОДА (priv_x)
   { "relu",      new() { ClForwardFunc = "nn_relu",      ClBackwardFormula =
```

```
"nn_relu_backward(priv_{0})" } },
    { "leakyrelu", new() { ClForwardFunc = "nn_leakyrelu", ClBackwardFormula =
"nn_leakyrelu_backward(priv_{0})" } },
    { "softplus",  new() { ClForwardFunc = "nn_softplus",  ClBackwardFormula =
"nn_softplus_backward(priv_{0})" } },
    { "sigmoid",   new() { ClForwardFunc = "nn_sigmoid",   ClBackwardFormula =
"nn_sigmoid_backward(priv_{0})" } },
    { "tanh",      new() { ClForwardFunc = "tanh",         ClBackwardFormula =
"nn_tanh_backward(priv_{0})" } },
    { "sin",       new() { ClForwardFunc = "sin",          ClBackwardFormula = "cos(priv_{0})" } },
    { "cos",       new() { ClForwardFunc = "cos",          ClBackwardFormula = "(-sin(priv_{0}))" } }
};


        public (string forwardSource, string backwardSource) TranslateWithGrad(
            string pseudoCode,
            Dictionary<string, float> constants,
            Dictionary<string, core.Tensor> tensors)
        {
            _externalConstants = constants ?? new Dictionary<string, float>();
            _tensorMeta = tensors ?? throw new ArgumentNullException(nameof(tensors));

            _bodyCode.Clear();
            _backwardBodyCode.Clear();
            DiscoveredTensors.Clear();
            OutputTensorNames.Clear(); // Очищаем список выходов перед новым проходом
            SyntaxTree tree = CSharpSyntaxTree.ParseText(pseudoCode);
            CompilationUnitSyntax root = tree.GetCompilationUnitRoot();
            Visit(root);

            string utilities = @"// Встроенные утилиты генерации шума и Dropout
inline float nn_rand(int tid, unsigned int seed) {
    unsigned int x = tid + seed;
    x = (x ^ 61) ^ (x >> 16);
    x += (x << 3);
    x = x ^ (x >> 4);
    x *= 0x27d4eb2d;
    x = x ^ (x >> 15);
    return (float)x / 4294967296.0f;
}
inline float nn_dropout(float x, float p, int tid, unsigned int seed) {
    if (p <= 0.0f) return x;
    if (p >= 1.0f) return 0.0f;
    float r = nn_rand(tid, seed);
    return (r < p) ? 0.0f : x / (1.0f - p);
}
inline float nn_dropout_backward(float grad_out, float p, int tid, unsigned int seed) {
    float r = nn_rand(tid, seed);
    return (r < p) ? 0.0f : grad_out / (1.0f - p);
}
```

```
// Добавляем к строке utilities в LocalRoslynTranspiler:

inline float nn_relu(float x) { return (x > 0.0f) ? x : 0.0f; }
inline float nn_relu_backward(float x) { return (x > 0.0f) ? 1.0f : 0.0f; }

inline float nn_leakyrelu(float x) { return (x > 0.0f) ? x : x * 0.01f; }
inline float nn_leakyrelu_backward(float x) { return (x > 0.0f) ? 1.0f : 0.01f; }

inline float nn_softplus(float x) { return log1p(exp(x)); }
inline float nn_softplus_backward(float x) { return 1.0f / (1.0f + exp(-x)); }

inline float nn_sigmoid(float x) { return 1.0f / (1.0f + exp(-x)); }
inline float nn_sigmoid_backward(float x) {
    float s = 1.0f / (1.0f + exp(-x));
    return s * (1.0f - s);
}

inline float nn_tanh_backward(float x) {
    float t = tanh(x);
    return 1.0f - t * t;
}




";

        // Вычисляем максимальный ранг (количество осей) среди всех обнаруженных тензоров
        var activeTensors = DiscoveredTensors.Select(name => _tensorMeta[name]).ToList();
        int maxRank = activeTensors.Max(t => t.Shape.Length);

        // За эталонную сетку берем форму самого большого тензора (у которого ранг максимален)
        var maxTensor = activeTensors.First(t => t.Shape.Length == maxRank);
        int[] maxShape = maxTensor.Shape;

        // --- ГЕНЕРАЦИЯ ДЕКОДЕРА КООРДИНАТ (Broadcasting Core) ---
        var coordDecoder = new StringBuilder();
        coordDecoder.Append("    // Восстановление многомерных координат из плоского tid\n");
        coordDecoder.Append($"    int rem = tid;\n");

        // Генерируем переменные координат: dim_0, dim_1...
        for (int i = maxRank - 1; i >= 0; i--)
        {
            if (i > 0)
            {
                coordDecoder.Append($"    int dim_{i} = rem % {maxShape[i]};\n");
                coordDecoder.Append($"    rem /= {maxShape[i]};\n");
            }
            else
```

```
            {
                coordDecoder.Append($"    int dim_{i} = rem;\n");
            }
        }
        coordDecoder.Append("\n");

        // --- ГЕНЕРАЦИЯ ИНДЕКСОВ ЧТЕНИЯ ДЛЯ КАЖДОГО ТЕНЗОРА ---
        var indexCalculators = new StringBuilder();
        indexCalculators.Append("    // Вычисление смещений с учетом правил Broadcasting\n");
        foreach (var name in DiscoveredTensors)
        {
            var t = _tensorMeta[name];
            indexCalculators.Append($"    int idx_{name} = 0;\n");

            // Выравниваем оси тензора по правому краю относительно максимального ранга
            int rankDiff = maxRank - t.Shape.Length;

            for (int i = 0; i < t.Shape.Length; i++)
            {
                int targetMaxDimIdx = i + rankDiff;

                // ПРАВИЛО BROADCASTING: если размерность оси равна 1,
                // то индекс по этой оси всегда 0, шаг зануляется (умножение на 0)
                if (t.Shape[i] == 1)
                {
                    indexCalculators.Append($"    idx_{name} += 0 * {t.Strides[i]};\n");
                }
                else
                {
                    // Иначе смещаемся на координату эталонной сетки, умноженную на stride оси
тензора
                    indexCalculators.Append($"    idx_{name} += dim_{targetMaxDimIdx} *
{t.Strides[i]};\n");
                }
            }
            // Защита от выхода за физические границы памяти
            indexCalculators.Append($"    idx_{name} = min(idx_{name}, {t.Size} - 1);\n\n");
        }

        // --- СБОРКА СТРОКИ ЯДРА FORWARD ---
        var fSrc = new StringBuilder(utilities);
        fSrc.Append("kernel void fused_elementwise_kernel(\n");
        BuildSignature(fSrc);
        //fSrc.Append("    int tid = get_global_id(0);\n\n");
        fSrc.Append("int tid=get_global_size(1)*get_global_id(0)+get_global_id(1);\n\n");//XY
оптимизация
        fSrc.Append(coordDecoder.ToString());
        fSrc.Append(indexCalculators.ToString());

        // Чтение в регистры по вычисленным индексам слияния размерностей
```

```
        fSrc.Append("    // Чтение во внутренние регистры\n");
        foreach (var name in DiscoveredTensors)
            fSrc.Append($"    float priv_{name} = {name}[idx_{name}];\n");

        fSrc.Append("\n    // Вычисления\n" + _bodyCode.ToString() + "\n");

        // Запись результатов во VRAM
        fSrc.Append("    // Сохранение результатов\n");
        foreach (var name in DiscoveredTensors)
        {
            fSrc.Append($"    if (tid < {name}_len) {{ {name}[idx_{name}] = priv_{name}; }}\n");
        }
        fSrc.Append("}");

        // --- СБОРКА СТРОКИ ЯДРА BACKWARD ---
        var bSrc = new StringBuilder(utilities);
        bSrc.Append("kernel void fused_elementwise_backward_kernel(\n");
        foreach (var name in DiscoveredTensors)
        {
            bSrc.Append($"    global float* {name}, global float* grad_{name}, global float*
out_grad_{name}, const int {name}_len,\n");
        }
        bSrc.Length -= 2;
        bSrc.Append(") {\n");
        //bSrc.Append("    int tid = get_global_id(0);\n\n");
        bSrc.Append("int tid=get_global_size(1)*get_global_id(0)+get_global_id(1);\n\n");//XY
оптимизация
        bSrc.Append(coordDecoder.ToString());
        bSrc.Append(indexCalculators.ToString());

        foreach (var name in DiscoveredTensors)
        {
            bSrc.Append($"    float priv_{name} = {name}[idx_{name}];\n");
            bSrc.Append($"    float priv_grad_{name} = grad_{name}[idx_{name}];\n");
            bSrc.Append($"    float priv_out_grad_{name} = 0.0f;\n");
        }
        bSrc.Append("\n    // Вычисление производных на регистрах\n");
        bSrc.Append(_backwardBodyCode.ToString());
        bSrc.Append("\n    // Сохранение рассчитанных градиентов\n");
        foreach (var name in DiscoveredTensors)
        {
            bSrc.Append($"    if (tid < {name}_len) {{ out_grad_{name}[idx_{name}] =
priv_out_grad_{name}; }}\n");
        }
        bSrc.Append("}");

        return (fSrc.ToString(), bSrc.ToString());
    }

    private void BuildSignature(StringBuilder sb)
    {
        int idx = 0;
```

```
            foreach (var name in DiscoveredTensors)
            {
                sb.Append($"    global float* {name}, const int {name}_len");
                if (idx < DiscoveredTensors.Count - 1) sb.Append(",\n");
                idx++;
            }
            sb.Append(") {\n");
        }


        /// <summary>
        /// Список имен тензоров, в которые производилась запись (левая часть оператора '=').
        /// </summary>
        public HashSet<string> OutputTensorNames { get; } = new
HashSet<string>(StringComparer.OrdinalIgnoreCase);


        public override SyntaxNode VisitAssignmentExpression(AssignmentExpressionSyntax node)
        {
            string leftVar = node.Left.ToString().Trim();
            DiscoveredTensors.Add(leftVar);

            // ФИКСАЦИЯ ВЫХОДА: Маркируем тензор как выходной, так как в него идет запись
            OutputTensorNames.Add(leftVar);

            string opStr = node.OperatorToken.Text;
            _bodyCode.Append($"    priv_{leftVar} {opStr} ");
            ProcessRightSide(node.Right, leftVar);
            _bodyCode.Append(";\n");

            return node;
        }

        private void ProcessRightSide(ExpressionSyntax expression, string leftVar)
        {
            if (expression is InvocationExpressionSyntax invocation)
            {
                // 1. ПЕРЕХВАТ ФУНКЦИЙ (tanh, sin, cos, dropout)
                string funcText = invocation.Expression.ToString();
                var firstArgExpr = invocation.ArgumentList.Arguments[0].Expression;
                string argName = firstArgExpr.ToString().Trim();

                DiscoveredTensors.Add(argName);

                /*if (funcText.Contains("tanh"))
                {
                    _bodyCode.Append($"tanh(priv_{argName})");
                    _backwardBodyCode.Append($"    priv_out_grad_{argName} += priv_grad_{leftVar}
* (1.0f - priv_{leftVar} * priv_{leftVar});\n");
                }
                else if (funcText.Contains("sin"))
                {
```

```
                _bodyCode.Append($"sin(priv_{argName})");
                _backwardBodyCode.Append($"    priv_out_grad_{argName} += priv_grad_{leftVar}
* cos(priv_{argName});\n");
            }
            else if (funcText.Contains("cos"))
            {
                _bodyCode.Append($"cos(priv_{argName})");
                _backwardBodyCode.Append($"    priv_out_grad_{argName} += priv_grad_{leftVar}
* (-sin(priv_{argName}));\n");
            }*/

            /*if (funcText.Contains("dropout"))
            {
                string pValStr = invocation.ArgumentList.Arguments[1].ToString();
                string seedValStr = invocation.ArgumentList.Arguments[2].ToString();

                _bodyCode.Append($"nn_dropout(priv_{argName}, {pValStr}f, tid, {seedValStr}u)");
                _backwardBodyCode.Append($"    priv_out_grad_{argName} +=
nn_dropout_backward(priv_grad_{leftVar}, {pValStr}f, tid, {seedValStr}u);\n");
            }*/



            // Специфичная логика для Dropout (так как у него 3 аргумента)
            if (funcText.Contains("dropout"))
            {
                string pValStr = invocation.ArgumentList.Arguments[1].ToString();
                string seedValStr = invocation.ArgumentList.Arguments[2].ToString();
                _bodyCode.Append($"nn_dropout(priv_{argName}, {pValStr}f, tid, {seedValStr}u)");
                _backwardBodyCode.Append($" priv_out_grad_{argName} +=
nn_dropout_backward(priv_grad_{leftVar}, {pValStr}f, tid, {seedValStr}u);\n");
                return;
            }

            // Универсальный конвейер для всех остальных активаций
            var matchedKey = Activations.Keys.FirstOrDefault(k => funcText.Contains(k));
            if (matchedKey != null)
            {
                var cfg = Activations[matchedKey];

                // Прямой проход: например, nn_sigmoid(priv_x)
                _bodyCode.Append($"{cfg.ClForwardFunc}(priv_{argName})");

                // Обратный проход: {0} заменяется строго на argName. leftVar все еще
передается,
                // но маппинг его игнорирует, сохраняя сигнатуру метода string.Format
стабильной.
                string backwardExpr = string.Format(cfg.ClBackwardFormula, argName, leftVar);
                _backwardBodyCode.Append($" priv_out_grad_{argName} += priv_grad_{leftVar} *
{backwardExpr};\n");
            }
```

```
        }
        else if (expression is BinaryExpressionSyntax binary)
        {
            // 2. ПЕРЕХВАТ БИНАРНОЙ МАТЕМАТИКИ (+, -, *, /)
            // Исправление 1: Рекурсивно раскрываем операнды, знаки подчеркивания
добавятся внутри IdentifierNameSyntax автоматически!
            ProcessRightSide(binary.Left, leftVar);

            string op = binary.OperatorToken.Text;
            _bodyCode.Append($" {op} ");

            ProcessRightSide(binary.Right, leftVar);

            string leftStr = binary.Left.ToString().Trim();
            string rightStr = binary.Right.ToString().Trim();

            if (op == "/" && _externalConstants.TryGetValue(rightStr, out float val))
            {
                string valStr = val.ToString("F1", System.Globalization.CultureInfo.InvariantCulture)
+ "f";
                _backwardBodyCode.Append($"    priv_out_grad_{leftStr} += priv_grad_{leftVar} /
{valStr};\n");
            }
            else if (op == "+" && !_externalConstants.ContainsKey(leftStr) && !
_externalConstants.ContainsKey(rightStr))
            {
                // Исправление 2: Добавлены потерянные знаки подчеркивания '_' для обратного
прохода сложения тензоров
                if (DiscoveredTensors.Contains(leftStr))
                    _backwardBodyCode.Append($"    priv_out_grad_{leftStr} +=
priv_grad_{leftVar};\n");

                if (DiscoveredTensors.Contains(rightStr))
                    _backwardBodyCode.Append($"    priv_out_grad_{rightStr} +=
priv_grad_{leftVar};\n");
            }
        }
        else if (expression is IdentifierNameSyntax identifier)
        {
            // 3. ПЕРЕХВАТ ИДЕНТИФИКАТОРОВ (Переменные-тензоры)
            string name = identifier.Identifier.Text.Trim();

            if (_externalConstants.TryGetValue(name, out float val))
            {
                _bodyCode.Append(val.ToString("F1",
System.Globalization.CultureInfo.InvariantCulture) + "f");
            }
            else
            {
                DiscoveredTensors.Add(name);
                // Исправление 3: Строго добавляем префикс со знаком подчеркивания для
прямого прохода
```

```
                _bodyCode.Append($"priv_{name}");
            }
        }
        else
        {
            // 4. ПРЯМЫЕ ЧИСЛОВЫЕ КОНСТАНТЫ ИЗ ТЕКСТА
            string numStr = expression.ToString().Trim().Replace(",", ".");
            if (!numStr.Contains(".")) numStr += ".0";
            if (!numStr.EndsWith("f") && !numStr.EndsWith("u")) numStr += "f";
            _bodyCode.Append(numStr);
        }
    }

  }
}
```

```
using System;
using cstorchcl.core;

namespace cstorchcl.compiler
{
    public class LocalVariable
    {
        public Tensor Tensor { get; }
        public string Name { get; }

        // Передаем метаданные формы для анализа Broadcasting
        public int[] Shape => Tensor.Shape;
        public int[] Strides => Tensor.Strides;

        public LocalVariable(Tensor tensor, string name)
        {
            Tensor = tensor ?? throw new ArgumentNullException(nameof(tensor));
            Name = name;
        }

        /// <summary>
        /// Эквивалент оператора присваивания (=) для обхода ограничений
Expression Trees.
        /// </summary>
        public LocalVariable Assign(LocalVariable value) => this;
        public LocalVariable Assign(double value) => this;

        // Перегруженные операторы теперь просто участвуют в построении
математического дерева
        public static LocalVariable operator +(LocalVariable a, double b) => a;
        public static LocalVariable operator -(LocalVariable a, double b) => a;
        public static LocalVariable operator *(LocalVariable a, double b) => a;
        public static LocalVariable operator /(LocalVariable a, double b) => a;

        public static LocalVariable operator +(LocalVariable a, LocalVariable b)
=> a;
        public static LocalVariable operator -(LocalVariable a, LocalVariable b)
=> a;
        public static LocalVariable operator *(LocalVariable a, LocalVariable b)
=> a;
        public static LocalVariable operator /(LocalVariable a, LocalVariable b)
=> a;
    }
```

```
    public static class nncl1oc
    {
        public static LocalVariable sin(LocalVariable v) => v;
        public static LocalVariable cos(LocalVariable v) => v;
        public static LocalVariable tanh(LocalVariable v) => v;
        public static LocalVariable dropout(LocalVariable v, float p, uint seed)
=> v;
        public static LocalVariable relu(LocalVariable v) => v;
        public static LocalVariable softplus(LocalVariable v) => v;
        public static LocalVariable leakyrelu(LocalVariable v) => v;

    }
}
```

```
namespace cstorchcl.compiler
{
   /// <summary>
   /// Диспетчер тяжелых глобальных операций библиотеки (Softmax, RMSNorm, BMM,
Permute, CausalConv).
   /// Загружает, компилирует и кэширует Си-ядра OpenCL.
   /// </summary>
   public class NnClGlob
   {
      private readonly ComputeContext _computeCtx;

      // Ссылки на скомпилированные программы
      private Program _prgRmsNorm;
      private Program _prgSoftmax;
      private Program _prgMatMul;
      private Program _prgPermute;
      private Program _prgConv;
      private Program _prgEmbedding;

      // Кэшированные скомпилированные ядра (экземпляры Kernel)
      private Kernel _kernelSoftmaxForward;
      private Kernel _kernelSoftmaxBackward;

      /// <summary>
      /// Инициализирует диспетчер ядер, привязываясь к активному ComputeContext.
      /// </summary>
      public NnClGlob(ComputeContext computeCtx)
      {
         _computeCtx = computeCtx ?? throw new ArgumentNullException(nameof(computeCtx));
         LoadKernels();
      }

      /// <summary>
      /// Считывает исходный Си-код ядер из папки cl_kernels и компилирует его.
      /// </summary>
      private void LoadKernels()
      {
         // Корневой путь к файлам ядер (предполагаем структуру проекта: папка cl_kernels
лежит рядом с бинарником или в корне)
```

```
        string kernelsDir = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
"cl_kernels");

        // Если запускаемся из среды разработки, поднимаемся выше до папки проекта
        if (!Directory.Exists(kernelsDir))
        {
            kernelsDir =
Path.GetFullPath(Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "..", "..", "..",
"cl_kernels"));
        }

        if (!Directory.Exists(kernelsDir))
            throw new DirectoryNotFoundException($"Директория с OpenCL ядрами не найдена
по пути: {kernelsDir}");

        var ctx = _computeCtx.Context;
        var dev = _computeCtx.Device;

        // 1. Загрузка и компиляция Softmax
        string softmaxCode = File.ReadAllText(Path.Combine(kernelsDir, "Softmax.cl"));
        _prgSoftmax = JITProgramCache.GetOrCompile(ctx, dev, softmaxCode);

        // Сразу кэшируем сами функции-ядра для вызова
        _kernelSoftmaxForward = _prgSoftmax.CreateKernel("softmax_forward");
        _kernelSoftmaxBackward = _prgSoftmax.CreateKernel("softmax_backward");

        // 2. Загрузка RMSNorm
        string rmsnormCode = File.ReadAllText(Path.Combine(kernelsDir, "rmsnorm.cl"));
        _prgRmsNorm = JITProgramCache.GetOrCompile(ctx, dev, rmsnormCode);
        _kernelRmsNormForward = _prgRmsNorm.CreateKernel("rmsnorm_forward");
        _kernelRmsNormBackward = _prgRmsNorm.CreateKernel("rmsnorm_backward");
        // 3. Загрузка Matmul / BMM
        string matmulCode = File.ReadAllText(Path.Combine(kernelsDir, "matmul.cl"));
        _prgMatMul = JITProgramCache.GetOrCompile(ctx, dev, matmulCode);
        _kernelBmmTiled = _prgMatMul.CreateKernel("bmm_tiled");
        // 4. Загрузка Permute
        string permuteCode = File.ReadAllText(Path.Combine(kernelsDir, "permute.cl"));
        _prgPermute = JITProgramCache.GetOrCompile(ctx, dev, permuteCode);
        _kernelPermute = _prgPermute.CreateKernel("tensor_permute");

        // 5. Загрузка CausalConv1d
        string convCode = File.ReadAllText(Path.Combine(kernelsDir, "CausalConv1d.cl"));
        _prgConv = JITProgramCache.GetOrCompile(ctx, dev, convCode);
        _kernelConvForward = _prgConv.CreateKernel("causal_conv1d_depthwise");
        _kernelConvBackwardInput = _prgConv.CreateKernel("causal_conv1d_backward_input");
        _kernelConvBackwardWeights =
_prgConv.CreateKernel("causal_conv1d_backward_weights");
        // 6. Загрузка Embedding
        string embeddingCode = File.ReadAllText(Path.Combine(kernelsDir, "embedding.cl"));
        _prgEmbedding = JITProgramCache.GetOrCompile(ctx, dev, embeddingCode);
        _kernelEmbeddingForward = _prgEmbedding.CreateKernel("embedding_forward");
        _kernelEmbeddingBackward = _prgEmbedding.CreateKernel("embedding_backward");
```

```
    }

    //
============================================================================
==
    // РЕАЛИЗАЦИЯ ОПЕРАЦИЙ (FORWARD PASS)
    //
============================================================================
==

    /// <summary>
    /// Глобальная безопасная операция Softmax (Forward).
    /// Рассчитана на работу со скорами внимания размерности [B * H * L, SEQ_LEN].
    /// </summary>
    public Tensor Softmax(Tensor x, int dim = -1)
    {
        // Эквивалент проверки размерности в Python: assert dim == -1 or dim == len(x.shape) -
1
        int targetDim = dim == -1 ? x.Shape.Length - 1 : dim;
        if (targetDim != x.Shape.Length - 1)
        {
            throw new NotSupportedException("Текущая реализация ядра Softmax поддерживает
редукцию только по последней оси.");
        }

        int seqLen = x.Shape[x.Shape.Length - 1]; // Последняя ось
        int numRows = x.Size / seqLen;           // Вычисляем количество строк

        // Создаем выходной тензор той же формы во VRAM (RequiresGrad пока передаем как
у исходного)
        Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: x.RequiresGrad);

        // Настройка сетки потоков (Сетка идентична вашей конфигурации на Python)
        IntPtr localSize = new IntPtr(256);
        IntPtr globalSize = new IntPtr(numRows * 256);

        // Получаем глобальную активную очередь выполнения задач из нашего реестра
        CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

        // Передаем аргументы в Си-ядро по порядку индексов (0, 1, 2, 3):
        // kernel void softmax_forward(global const float* X, global float* Y, const int seq_len)
        _kernelSoftmaxForward.Arguments[0].SetValue(x.DataBuffer);
        _kernelSoftmaxForward.Arguments[1].SetValue(y.DataBuffer);
        _kernelSoftmaxForward.Arguments[2].SetValue(seqLen);

        // Запуск выполнения ядра на GPU
        activeQueue.EnqueueNDRangeKernel(_kernelSoftmaxForward, null, new[] { globalSize },
new[] { localSize });

        // Синхронизация: гарантирует завершение вычислений перед чтением
        activeQueue.Finish();
```

```
            // Если включен режим Autograd и требуется расчет градиента, привязываем контекст создателя к тензору
            if (y.RequiresGrad)
            {
                // Создаем операцию, передавая входы и выходы
                y.Creator = new SoftmaxOp(x, y, this);
            }

            return y;
        }
        /// <summary>
        /// Обратный проход глобальной операции Softmax.
        /// Использует уже вычисленный forward-выход тензора Y для экономии памяти VRAM.
        /// </summary>
        /// <param name="gradOutput">Пришедший сверху градиент dL/dY [B * H * L, SEQ_LEN]</param>
        /// <param name="y">Исходный выход прямого прохода Softmax (вероятности) [B * H * L, SEQ_LEN]</param>
        /// <returns>Выходной градиент по входу dL/dX [B * H * L, SEQ_LEN]</returns>
        public Tensor SoftmaxBackward(Tensor gradOutput, Tensor y)
        {
            int seqLen = y.Shape[y.Shape.Length - 1];
            int numRows = y.Size / seqLen;

            // Выделяем тензор для результирующего градиента по входу (по форме совпадает с Y)
            Tensor gradInput = new Tensor(y.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);

            // Настройка сетки потоков (идентично forward шагу)
            IntPtr localSize = new IntPtr(256);
            IntPtr globalSize = new IntPtr(numRows * 256);

            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
            int t = 0;
            // Передаем аргументы в Си-ядро softmax_backward:
            // kernel void softmax_backward(global const float* grad_output, global const float* Y, global float* grad_input, const int seq_len)
            _kernelSoftmaxBackward.Arguments[t++].SetValue(gradOutput.DataBuffer);
            _kernelSoftmaxBackward.Arguments[t++].SetValue(y.DataBuffer);
            _kernelSoftmaxBackward.Arguments[t++].SetValue(gradInput.DataBuffer);
            _kernelSoftmaxBackward.Arguments[t++].SetValue(seqLen);

            // Запуск ядра
            activeQueue.EnqueueNDRangeKernel(_kernelSoftmaxBackward, null, new[] { globalSize }, new[] { localSize });
            activeQueue.Finish();

            return gradInput;
        }
        #region RSNORM
```

```
        // Добавьте эти переменные в начало класса NnClGlob к остальным ядрам:
        private Kernel _kernelRmsNormForward;
        private Kernel _kernelRmsNormBackward;

        // Добавьте их инициализацию в метод LoadKernels():
        // _kernelRmsNormForward = _prgRmsNorm.CreateKernel("rmsnorm_forward");
        // _kernelRmsNormBackward = _prgRmsNorm.CreateKernel("rmsnorm_backward");

        /// <summary>
        /// Глобальная операция RMSNorm (Forward).
        /// Обрабатывает тензор любой формы, выполняя нормализацию по последней оси
d_model.
        /// </summary>
        public Tensor RmsNorm(Tensor x, Tensor gamma)
        {
            int dModel = x.Shape[x.Shape.Length - 1]; // Последняя ось
            int numRows = x.Size / dModel;           // Количество строк (B * L)

            // Выходной тензор
            Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: x.RequiresGrad || gamma.RequiresGrad);

            // Буфер для кэша инвертированного RMS (нужен для Backward)
            // Размер: numRows * sizeof(float)
            NOpenCL.Buffer invRmsCache =
_computeCtx.Context.CreateBuffer(MemoryFlags.ReadWrite, numRows * sizeof(float));

            // Настройка сетки потоков
            IntPtr localSize = new IntPtr(256);
            IntPtr globalSize = new IntPtr(numRows * 256);

            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
            int t = 0;
            // Передаем аргументы ядра:
            // kernel void rmsnorm_forward(global const float* X, global const float* gamma, global
float* Y, global float* inv_rms_cache, const int d_model)
            _kernelRmsNormForward.Arguments[t++].SetValue(x.DataBuffer);
            _kernelRmsNormForward.Arguments[t++].SetValue(gamma.DataBuffer);
            _kernelRmsNormForward.Arguments[t++].SetValue(y.DataBuffer);
            _kernelRmsNormForward.Arguments[t++].SetValue(invRmsCache);
            _kernelRmsNormForward.Arguments[t++].SetValue(dModel);

            activeQueue.EnqueueNDRangeKernel(_kernelRmsNormForward, null, new[]
{ globalSize }, new[] { localSize });
            activeQueue.Finish();

            // Если обучаемся, сохраняем кэш слоя в GradientStorage
            if (y.RequiresGrad)
            {
                var buffers = new LayerBuffers();
                buffers.layerBuffers.Add("inv_rms_cache", invRmsCache);
                GradientStorage.Push(buffers);
```

```
            // Привязываем узел операции к графу
            y.Creator = new RMSNormOp(x, gamma, y, this);
        }
        else
        {
            // Если обучение не идет, кэш нам больше не нужен, сразу освобождаем VRAM
            invRmsCache.Dispose();
        }

        return y;
    }

    /// <summary>
    /// Обратный проход глобальной операции RMSNorm.
    /// Вычисляет градиенты по входу X и по весам Gamma.
    /// </summary>
    public unsafe (Tensor gradX, Tensor gradGamma) RmsNormBackward(Tensor gradOutput,
Tensor x, Tensor gamma, NOpenCL.Buffer invRmsCache)
    {
        int dModel = x.Shape[x.Shape.Length - 1];
        int numRows = x.Size / dModel;

        // Выделяем выходные градиенты
        Tensor gradX = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: false);
        Tensor gradGamma = new Tensor(gamma.Shape, _computeCtx.Context,
MemoryFlags.ReadWrite, requiresGrad: false);

        // Промежуточный буфер градиентов по весам для каждой строки (для последующей
редукции)
        long partSizeInBytes = (long)x.Size * sizeof(float);
        using NOpenCL.Buffer gradGammaPart =
_computeCtx.Context.CreateBuffer(MemoryFlags.ReadWrite, partSizeInBytes);

        IntPtr localSize = new IntPtr(256);
        IntPtr globalSize = new IntPtr(numRows * 256);

        CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
        int t = 0;
        // Передаем аргументы ядра:
        // kernel void rmsnorm_backward(global const float* grad_output, global const float* X,
global const float* gamma, global const float* inv_rms_cache, global float* grad_input, global
float* grad_gamma_part, const int d_model)
        _kernelRmsNormBackward.Arguments[t++].SetValue(gradOutput.DataBuffer);
        _kernelRmsNormBackward.Arguments[t++].SetValue(x.DataBuffer);
        _kernelRmsNormBackward.Arguments[t++].SetValue(gamma.DataBuffer);
        _kernelRmsNormBackward.Arguments[t++].SetValue(invRmsCache);
        _kernelRmsNormBackward.Arguments[t++].SetValue(gradX.DataBuffer);
        _kernelRmsNormBackward.Arguments[t++].SetValue(gradGammaPart);
        _kernelRmsNormBackward.Arguments[t++].SetValue(dModel);
```

```
            activeQueue.EnqueueNDRangeKernel(_kernelRmsNormBackward, null, new[] { globalSize
}, new[] { localSize });
            activeQueue.Finish();

            // --- РЕДУКЦИЯ ГРАДИЕНТОВ ПО ВЕСАМ (Адаптация кода с numpy на C#) ---
            // Считываем частичные градиенты во временный массив на CPU для суммирования
строк
            float[] hostParts = new float[numRows * dModel];
            fixed (float* hp = hostParts)
            {
                activeQueue.EnqueueReadBuffer(gradGammaPart, true, 0, hostParts.Length *
sizeof(float), (IntPtr)hp);
            }
            activeQueue.Finish();

            // Суммируем градиенты от всех токенов/строк в единый вектор весов Gamma
            float[] finalGradGamma = new float[dModel];
            for (int r = 0; r < numRows; r++)
            {
                int rowOffset = r * dModel;
                for (int c = 0; c < dModel; c++)
                {
                    finalGradGamma[c] += hostParts[rowOffset + c];
                }
            }

            // Загружаем финальный саккумулированный вектор градиента обратно во VRAM
буфер gradGamma
            fixed (float* fgg = finalGradGamma)
            {
                activeQueue.EnqueueWriteBuffer(gradGamma.DataBuffer, true, 0,
finalGradGamma.Length * sizeof(float), (IntPtr)fgg);
            }
            activeQueue.Finish();

            return (gradX, gradGamma);
        }

        #endregion
        #region Permute
        // Добавьте эту переменную в начало класса NnClGlob к остальным ядрам:
        private Kernel _kernelPermute;

        // Добавьте её инициализацию в метод LoadKernels():
        // _kernelPermute = _prgPermute.CreateKernel("tensor_permute");

        /// <summary>
        /// Глобальная перестановка осей тензора (Аналог x.permute(...) или x.transpose(...)).
        /// Поддерживает тензоры любой размерности.
        /// </summary>
        public unsafe Tensor Permute(Tensor x, int[] axes)
        {
```

```
        if (axes.Length != x.Shape.Length)
            throw new ArgumentException("Количество осей перестановки должно совпадать с размерностью тензора.");

        // Вычисляем новую форму тензора на основе массива перестановки осей
        int[] dstShape = new int[axes.Length];
        for (int i = 0; i < axes.Length; i++)
        {
            dstShape[i] = x.Shape[axes[i]];
        }

        Tensor y = new Tensor(dstShape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: x.RequiresGrad);

        // Подготовка метаданных для Си-ядра: размеры осей во VRAM буферы int32
        int[] srcDims = x.Shape;
        int[] dstDims = dstShape;
        fixed (int* ax = axes, srd = srcDims, dstd = dstDims)
        {
            Context ctx = _computeCtx.Context;
            using NOpenCL.Buffer srcDimsBuf = ctx.CreateBuffer(MemoryFlags.ReadOnly | MemoryFlags.UseHostPointer, srcDims.Length * sizeof(int), (IntPtr)srd);
            using NOpenCL.Buffer dstDimsBuf = ctx.CreateBuffer(MemoryFlags.ReadOnly | MemoryFlags.UseHostPointer, dstDims.Length * sizeof(int), (IntPtr)dstd);
            using NOpenCL.Buffer axesBuf = ctx.CreateBuffer(MemoryFlags.ReadOnly | MemoryFlags.UseHostPointer, axes.Length * sizeof(int), (IntPtr)ax);

            IntPtr globalSize = new IntPtr(x.Size);
            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
            int t = 0;
            // Передаем аргументы в Си-ядро tensor_permute:
            // kernel void tensor_permute(global const float* src, global float* dst, global const int* src_dims, global const int* dst_dims, global const int* permute_axes, const int num_dims, const int total_elements)
            _kernelPermute.Arguments[t++].SetValue(x.DataBuffer);
            _kernelPermute.Arguments[t++].SetValue(y.DataBuffer);
            _kernelPermute.Arguments[t++].SetValue(srcDimsBuf);
            _kernelPermute.Arguments[t++].SetValue(dstDimsBuf);
            _kernelPermute.Arguments[t++].SetValue(axesBuf);
            _kernelPermute.Arguments[t++].SetValue(x.Shape.Length);
            _kernelPermute.Arguments[t++].SetValue(x.Size);

            activeQueue.EnqueueNDRangeKernel(_kernelPermute, new[] { globalSize }, null);
            activeQueue.Finish();
        }
        return y;
    }

    #endregion
    #region BMM
    // Добавьте эту переменную в начало класса NnClGlob к остальным ядрам:
    private Kernel _kernelBmmTiled;
```

```
        // Добавьте её инициализацию в метод LoadKernels():
        // _kernelBmmTiled = _prgMatMul.CreateKernel("bmm_tiled");

        /// <summary>
        /// Пакетное матричное умножение (Batch Matrix Multiplication).
        /// Ожидает строго 3D-тензоры: A [Batch, M, K], B [Batch, K, N] -> Выход [Batch, M, N].
        /// </summary>
        /// <summary>
        /// Пакетное матричное умножение с поддержкой Batch Broadcasting для весов модели.
        /// </summary>
        public Tensor Bmm(Tensor a, Tensor b)
        {
            if (a.Shape.Length != 3 || b.Shape.Length != 3)
                throw new ArgumentException("BMM ожидает строго 3D-тензоры формы [Batch, M, K].");

            // ПРАВИЛО BROADCASTING ДЛЯ БАТЧА:
            // Если батч у B равен 1, а у A больше 1, то разрешаем операцию (веса применятся ко всем примерам)
            if (a.Shape[0] != b.Shape[0] && b.Shape[0] != 1)
                throw new ArgumentException($"Несовпадение батч-размерностей: {a.Shape[0]} и {b.Shape[0]}.");

            if (a.Shape[2] != b.Shape[1])
                throw new ArgumentException($"Несовпадение внутренних осей матриц: K-ось {a.Shape[2]} != {b.Shape[1]}.");

            int batch = a.Shape[0]; // Итоговый батч всегда равен батчу данных (3)
            int m = a.Shape[1];
            int k = a.Shape[2];
            int n = b.Shape[2];

            int[] outShape = { batch, m, n };
            Tensor c = new Tensor(outShape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: a.RequiresGrad || b.RequiresGrad);

            int batchStrideA = m * k;

            // ВАЖНОЕ АРХИТЕКТУРНОЕ ИСПРАВЛЕНИЕ:
            // Если у тензора B (веса) батч равен 1, то шаг смещения пакета во VRAM принудительно зануляется!
            // Ядро на GPU будет читать одну и ту же матрицу для каждого шага батча
            int batchStrideB = (b.Shape[0] == 1) ? 0 : (k * n);

            int batchStrideC = m * n;

            // Настройка 3D-сетки потоков под тайлы 16x16
            int gx = (int)Math.Ceiling(n / 16.0) * 16;
            int gy = (int)Math.Ceiling(m / 16.0) * 16;
            int gz = batch;
```

```
            IntPtr localSizeX = new IntPtr(16);
            IntPtr localSizeY = new IntPtr(16);
            IntPtr localSizeZ = new IntPtr(1);

            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
            int t = 0;
            _kernelBmmTiled.Arguments[t++].SetValue(a.DataBuffer);
            _kernelBmmTiled.Arguments[t++].SetValue(b.DataBuffer);
            _kernelBmmTiled.Arguments[t++].SetValue(c.DataBuffer);
            _kernelBmmTiled.Arguments[t++].SetValue(m);
            _kernelBmmTiled.Arguments[t++].SetValue(k);
            _kernelBmmTiled.Arguments[t++].SetValue(n);
            _kernelBmmTiled.Arguments[t++].SetValue(batchStrideA);
            _kernelBmmTiled.Arguments[t++].SetValue(batchStrideB); // Передаем
скорректированный шаг!
            _kernelBmmTiled.Arguments[t++].SetValue(batchStrideC);

            activeQueue.EnqueueNDRangeKernel(
                _kernelBmmTiled,
                null,
                new[] { new IntPtr(gx), new IntPtr(gy), new IntPtr(gz) },
                new[] { localSizeX, localSizeY, localSizeZ }
            );
            activeQueue.Finish();

            if (c.RequiresGrad)
            {
                c.Creator = new BmmOp(a, b, c, this);
            }

            return c;
        }


        #endregion
        #region CasualConv1d
        // Добавьте эти переменные в начало класса NnClGlob к остальным ядрам:
        private Kernel _kernelConvForward;
        private Kernel _kernelConvBackwardInput;
        private Kernel _kernelConvBackwardWeights;

        // Добавьте их инициализацию в метод LoadKernels():
        // _kernelConvForward = _prgConv.CreateKernel("causal_conv1d_depthwise");
        // _kernelConvBackwardInput = _prgConv.CreateKernel("causal_conv1d_backward_input");
        // _kernelConvBackwardWeights =
_prgConv.CreateKernel("causal_conv1d_backward_weights");

        /// <summary>
        /// Каузальная одномерная Depthwise свёртка (Forward).
        /// Форма X: [Batch, Channels, Length]
        /// Форма Weight: [Channels, KernelSize]
        /// Форма Bias: [Channels]
```

```
        /// </summary>
        public Tensor CausalConv1d(Tensor x, Tensor weight, Tensor bias)
        {
            if (x.Shape.Length != 3)
                throw new ArgumentException("Входной тензор X должен иметь 3 измерения [Batch, Channels, Length].");

            int batch = x.Shape[0];
            int channels = x.Shape[1];
            int length = x.Shape[2];
            int kernelSize = weight.Shape[1];

            Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
                requiresGrad: x.RequiresGrad || weight.RequiresGrad || bias.RequiresGrad);

            // Настройка 3D сетки потоков по осям временной последовательности, каналов и батча
            IntPtr globalSizeX = new IntPtr(length);
            IntPtr globalSizeY = new IntPtr(channels);
            IntPtr globalSizeZ = new IntPtr(batch);

            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
            int t = 0;
            // Передаем аргументы ядра causal_conv1d_depthwise
            _kernelConvForward.Arguments[t++].SetValue(x.DataBuffer);
            _kernelConvForward.Arguments[t++].SetValue(weight.DataBuffer);
            _kernelConvForward.Arguments[t++].SetValue(bias.DataBuffer);
            _kernelConvForward.Arguments[t++].SetValue(y.DataBuffer);
            _kernelConvForward.Arguments[t++].SetValue(batch);
            _kernelConvForward.Arguments[t++].SetValue(channels);
            _kernelConvForward.Arguments[t++].SetValue(length);
            _kernelConvForward.Arguments[t++].SetValue(kernelSize);

            activeQueue.EnqueueNDRangeKernel(
                _kernelConvForward,

                new[] { globalSizeX, globalSizeY, globalSizeZ },
                null
            );
            activeQueue.Finish();

            if (y.RequiresGrad)
            {
                y.Creator = new CausalConv1dOp(x, weight, bias, y, this);
            }

            return y;
        }

        /// <summary>
        /// Обратный проход каузальной свёртки. Вычисляет градиенты по входу, весам и смещению.
```

```
    /// </summary>
    public (Tensor gradX, Tensor gradWeight, Tensor gradBias) CausalConv1dBackward(Tensor gradOutput, Tensor x, Tensor weight)
    {
        int batch = x.Shape[0];
        int channels = x.Shape[1];
        int length = x.Shape[2];
        int kernelSize = weight.Shape[1];

        // Аллокация результирующих тензоров градиентов
        Tensor gradX = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);
        Tensor gradWeight = new Tensor(weight.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);
        Tensor gradBias = new Tensor(new[] { channels }, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);

        CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

        // 1. Запуск ядра расчета градиента по входу X (Сетка 3D: Length, Channels, Batch)
        IntPtr gSizeInX = new IntPtr(length);
        IntPtr gSizeInY = new IntPtr(channels);
        IntPtr gSizeInZ = new IntPtr(batch);
        int t = 0;
        _kernelConvBackwardInput.Arguments[t++].SetValue(gradOutput.DataBuffer);
        _kernelConvBackwardInput.Arguments[t++].SetValue(weight.DataBuffer);
        _kernelConvBackwardInput.Arguments[t++].SetValue(gradX.DataBuffer);
        _kernelConvBackwardInput.Arguments[t++].SetValue(batch);
        _kernelConvBackwardInput.Arguments[t++].SetValue(channels);
        _kernelConvBackwardInput.Arguments[t++].SetValue(length);
        _kernelConvBackwardInput.Arguments[t++].SetValue(kernelSize);

        activeQueue.EnqueueNDRangeKernel(
            _kernelConvBackwardInput,

            new[] { gSizeInX, gSizeInY, gSizeInZ },
            null
        );

        // 2. Запуск ядра расчета градиентов по параметрам Weight и Bias (Сетка 2D: KernelSize, Channels)
        IntPtr gSizeW_X = new IntPtr(kernelSize);
        IntPtr gSizeW_Y = new IntPtr(channels);
        t=0;
        _kernelConvBackwardWeights.Arguments[t++].SetValue(gradOutput.DataBuffer);
        _kernelConvBackwardWeights.Arguments[t++].SetValue(x.DataBuffer);
        _kernelConvBackwardWeights.Arguments[t++].SetValue(gradWeight.DataBuffer);
        _kernelConvBackwardWeights.Arguments[t++].SetValue(gradBias.DataBuffer);
        _kernelConvBackwardWeights.Arguments[t++].SetValue(batch);
        _kernelConvBackwardWeights.Arguments[t++].SetValue(channels);
        _kernelConvBackwardWeights.Arguments[t++].SetValue(length);
        _kernelConvBackwardWeights.Arguments[t++].SetValue(kernelSize);
```

```
            activeQueue.EnqueueNDRangeKernel(
                _kernelConvBackwardWeights,

                new[] { gSizeW_X, gSizeW_Y },
                null
            );

            // Дожидаемся окончания работы обоих ядер на GPU
            activeQueue.Finish();

            return (gradX, gradWeight, gradBias);
        }

        #endregion
        #region Embedding
        // Добавьте эти переменные в начало класса NnClGlob к остальным ядрам:
        private Kernel _kernelEmbeddingForward;
        private Kernel _kernelEmbeddingBackward;

        // Добавьте их инициализацию в метод LoadKernels():
        // _kernelEmbeddingForward = _prgEmbedding.CreateKernel("embedding_forward");
        // _kernelEmbeddingBackward = _prgEmbedding.CreateKernel("embedding_backward");

        /// <summary>
        /// Выборка эмбеддингов по ID токенов (Forward).
        /// </summary>
        /// <param name="xTokensBuf">Низкоуровневый OpenCL буфер, хранящий целочисленные ID токенов (int32).</param>
        /// <param name="weight">Тензор весов эмбеддингов формы [VocabSize, EmbeddingDim].</param>
        /// <param name="totalTokens">Общее количество токенов в батче (Batch * SeqLen).</param>
        /// <returns>Выходной тензор формы [Batch * SeqLen, EmbeddingDim] или эквивалентной многомерной формы.</returns>
        public Tensor Embedding(NOpenCL.Buffer xTokensBuf, Tensor weight, int totalTokens)
        {
            if (weight.Shape.Length != 2)
                throw new ArgumentException("Матрица весов Embedding должна быть строго 2D тензором [VocabSize, EmbeddingDim].");

            int vocabSize = weight.Shape[0];
            int embeddingDim = weight.Shape[1];

            // Создаем выходной тензор. Форма: [TotalTokens, EmbeddingDim]
            int[] outShape = { totalTokens, embeddingDim };
            Tensor y = new Tensor(outShape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: weight.RequiresGrad);

            // Настройка 2D-сетки потоков: по количеству токенов и по размерности эмбеддинга
            IntPtr globalSizeX = new IntPtr(totalTokens);
            IntPtr globalSizeY = new IntPtr(embeddingDim);
```

```
            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
            int t = 0;
            // Передаем аргументы в Си-ядро embedding_forward:
            // kernel void embedding_forward(global const int* X_tokens, global const float* weight,
global float* Y_out, const int embedding_dim, const int total_tokens)
            _kernelEmbeddingForward.Arguments[t++].SetValue(xTokensBuf);
            _kernelEmbeddingForward.Arguments[t++].SetValue(weight.DataBuffer);
            _kernelEmbeddingForward.Arguments[t++].SetValue(y.DataBuffer);
            _kernelEmbeddingForward.Arguments[t++].SetValue(embeddingDim);
            _kernelEmbeddingForward.Arguments[t++].SetValue(totalTokens);

            activeQueue.EnqueueNDRangeKernel(
                _kernelEmbeddingForward,

                new[] { globalSizeX, globalSizeY },
                null
            );
            activeQueue.Finish();

            // Если матрица весов требует градиентов, регистрируем операцию в графе
            if (y.RequiresGrad)
            {
                y.Creator = new EmbeddingOp(xTokensBuf, weight, y, totalTokens, this);
            }

            return y;
        }

        /// <summary>
        /// Обратный проход слоя Embedding. Рассчитывает градиент по матрице весов
(gradWeight).
        /// Альтернативная реализация: обнуление буфера через запись host-массива (замена
отсутствующего FillBuffer).
        /// </summary>
        public unsafe Tensor EmbeddingBackward(Tensor gradOutput, NOpenCL.Buffer xTokensBuf,
Tensor weight, int totalTokens)
        {
            int vocabSize = weight.Shape[0];
            int embeddingDim = weight.Shape[1];

            // Выделяем тензор для накопления градиентов по весам (форма совпадает с Weight)
            Tensor gradWeight = new Tensor(weight.Shape, _computeCtx.Context,
MemoryFlags.ReadWrite, requiresGrad: false);

            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

            //
================================================================
==
            // ИСПРАВЛЕНИЕ: ОБНУЛЕНИЕ БУФЕРА ЧЕРЕЗ ХОСТ-МАССИВ СИЛАМИ .NET
```

```
        //
================================================================================
==
        // Создаем массив нулей на CPU. В C# при выделении массива float[]
        // среда выполнения .NET гарантирует, что он изначально полностью заполнен
нулями.
        float[] zeroHostPattern = new float[gradWeight.Size];

        // Вычисляем размер буфера в байтах
        long bufferSizeInBytes = (long)gradWeight.Size * sizeof(float);
        fixed (float* zhp = zeroHostPattern)
        {
            // Синхронно записываем массив нулей во VRAM девайса, чтобы очистить старые
данные
            // Аргументы: буфер, blocking_write (true), смещение, размер в байтах, массив-
источник
            activeQueue.EnqueueWriteBuffer(gradWeight.DataBuffer, true, 0, bufferSizeInBytes,
(IntPtr)zhp);
        }
        //
================================================================================
==

        // Настройка 2D-сетки потоков под конфигурацию обратного прохода
        IntPtr globalSizeX = new IntPtr(totalTokens);
        IntPtr globalSizeY = new IntPtr(embeddingDim);
        int t = 0;
        // Передаем аргументы в Си-ядро embedding_backward:
        // kernel void embedding_backward(global const float* grad_output, global const int*
X_tokens, global float* grad_weight, const int embedding_dim, const int total_tokens)
        _kernelEmbeddingBackward.Arguments[t++].SetValue(gradOutput.DataBuffer);
        _kernelEmbeddingBackward.Arguments[t++].SetValue(xTokensBuf);
        _kernelEmbeddingBackward.Arguments[t++].SetValue(gradWeight.DataBuffer);
        _kernelEmbeddingBackward.Arguments[t++].SetValue(embeddingDim);
        _kernelEmbeddingBackward.Arguments[t++].SetValue(totalTokens);

        // Запускаем атомарное ядро
        activeQueue.EnqueueNDRangeKernel(
            _kernelEmbeddingBackward,

            new[] { globalSizeX, globalSizeY },
            null
        );
        activeQueue.Finish();

        return gradWeight;
    }


    #endregion
#if DEBUG
    /// <summary>
```

```
        /// Временная отладочная операция ELU (Forward). Код ядра встроен прямо в метод.
        /// </summary>
        public Tensor Elu(Tensor x, float alpha = 1.0f)
        {
            Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: x.RequiresGrad);

            string kernelSource = @"
                kernel void elu_forward(global const float* X, global float* Y) {
                    int id = get_global_id(0);
                    float val = X[id];
                    Y[id] = (val > 0.0f) ? val :"+ alpha.ToString().Replace(',','.')+@" * (exp(val) - 1.0f);
                }";

            // Используем наш JITProgramCache для предотвращения повторной компиляции
            var prg = JITProgramCache.GetOrCompile(_computeCtx.Context, _computeCtx.Device,
kernelSource);
            using var kernel = prg.CreateKernel("elu_forward");

            kernel.Arguments[0].SetValue(x.DataBuffer);
            kernel.Arguments[1].SetValue(y.DataBuffer);
            //kernel.Arguments[2].SetValue(alpha);

            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
            activeQueue.EnqueueNDRangeKernel(kernel,  new[] { new IntPtr(x.Size) }, null);
            activeQueue.Finish();

            if (y.RequiresGrad)
            {
                y.Creator = new EluOp(x, y, alpha, this);
            }

            return y;
        }

        /// <summary>
        /// Временная отладочная операция ELU (Backward).
        /// </summary>
        public Tensor EluBackward(Tensor gradOutput, Tensor y, float alpha = 1.0f)
        {
            Tensor gradInput = new Tensor(y.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: false);

            string kernelSource = @"
                kernel void elu_backward(global const float* grad_output, global const float* Y, global
float* grad_input) {
                    int id = get_global_id(0);
                    float y_val = Y[id];
                    grad_input[id] = (y_val > 0.0f) ? grad_output[id] : grad_output[id] * (y_val +
"+alpha.ToString().Replace(',','.')+@");
                }";
```

```
            var prg = JITProgramCache.GetOrCompile(_computeCtx.Context, _computeCtx.Device,
kernelSource);
            using var kernel = prg.CreateKernel("elu_backward");

            kernel.Arguments[0].SetValue(gradOutput.DataBuffer);
            kernel.Arguments[1].SetValue(y.DataBuffer);
            kernel.Arguments[2].SetValue(gradInput.DataBuffer);
            //kernel.Arguments[3].SetValue(alpha);

            CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
            activeQueue.EnqueueNDRangeKernel(kernel,  new[] { new IntPtr(y.Size) }, null);
            activeQueue.Finish();

            return gradInput;
        }
#endif
        #region MSELoss
        /// <summary>
        /// Вычисляет MSE Loss и возвращает скаляр ошибки, регистрируя шаг в Autograd.
        /// </summary>
        public Tensor MseLoss(Tensor pred, Tensor target)
        {
            Tensor loss = new Tensor(new[] { 1 }, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: pred.RequiresGrad);

            // Для простоты посчитаем разницу прямо на GPU
            string kernelSource = @"
                kernel void mse_forward(global const float* P, global const float* T, global float* L,
const int size) {
                    float sum = 0.0f;
                    for(int i = 0; i < size; i++) {
                        float diff = P[i] - T[i];
                        sum += diff * diff;
                    }
                    L[0] = sum / size;
                }";

            var prg = JITProgramCache.GetOrCompile(_computeCtx.Context, _computeCtx.Device,
kernelSource);
            using var kernel = prg.CreateKernel("mse_forward");
            kernel.Arguments[0].SetValue(pred.DataBuffer);
            kernel.Arguments[1].SetValue(target.DataBuffer);
            kernel.Arguments[2].SetValue(loss.DataBuffer);
            kernel.Arguments[3].SetValue(pred.Size);

            CommandQueue queue = DeviceQueueRegistry.GetCurrent();
            queue.EnqueueNDRangeKernel(kernel,  new[] { new IntPtr(1) }, null);
            queue.Finish();

            if (loss.RequiresGrad)
            {
                loss.Creator = new MseLossOp(pred, target, loss, this);
```

```
            }

            return loss;
        }

        /// <summary>
        /// Градиент для MSE Loss: dL/dPred = 2 * (Pred - Target) / Size
        /// </summary>
        public Tensor MseLossBackward(Tensor pred, Tensor target)
        {
            Tensor gradInput = new Tensor(pred.Shape, _computeCtx.Context,
MemoryFlags.ReadWrite, requiresGrad: false);

            string kernelSource = @"
                kernel void mse_backward(global const float* P, global const float* T, global float* GI,
const int size) {
                    int id = get_global_id(0);
                    GI[id] = 2.0f * (P[id] - T[id]) / size;
                }";

            var prg = JITProgramCache.GetOrCompile(_computeCtx.Context, _computeCtx.Device,
kernelSource);
            using var kernel = prg.CreateKernel("mse_backward");
            kernel.Arguments[0].SetValue(pred.DataBuffer);
            kernel.Arguments[1].SetValue(target.DataBuffer);
            kernel.Arguments[2].SetValue(gradInput.DataBuffer);
            kernel.Arguments[3].SetValue(pred.Size);

            CommandQueue queue = DeviceQueueRegistry.GetCurrent();
            queue.EnqueueNDRangeKernel(kernel,  new[] { new IntPtr(pred.Size) }, null);
            queue.Finish();

            return gradInput;
        }

        #endregion
        public Dictionary<string, Tensor> RunLocal(string pseudoCode, Dictionary<string, Tensor>
tensors, Dictionary<string, float> constants = null)
        {
            return NnClLoc.Run(pseudoCode, tensors, _computeCtx, constants);
        }

    }
}
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using NOpenCL;
using cstorchcl.core;

namespace cstorchcl.compiler
{
    public static class NnClLoc
    {
        /// <summary>
        /// Выполняет динамическую сборку и запуск сплавленного OpenCL-ядра на
2D-сетке варпов.
        /// </summary>
        ///         /// <summary>
        /// Выполняет динамическую сборку и возвращает СЛОВАРЬ новых выходных
тензоров.
        /// </summary>
        public static Dictionary<string, Tensor> Run(string pseudoCode,
Dictionary<string, Tensor> tensors, ComputeContext computeCtx,
Dictionary<string, float> constants = null)
        {
            if (string.IsNullOrWhiteSpace(pseudoCode)) return null;
            if (tensors == null || tensors.Count == 0) return null;

            var transpiler = new LocalRoslynTranspiler();
            var (forwardSource, backwardSource) =
transpiler.TranslateWithGrad(pseudoCode, constants, tensors);

            int N = tensors.Values.Max(t => t.Size);
            CommandQueue queue = DeviceQueueRegistry.GetCurrent();
            Context context = queue.Context;

            //
============================================================================
            // ИСПРАВЛЕНИЕ: Выделяем новые изолированные буферы для результатов
вычислений
            //
============================================================================
            var outputDict = new Dictionary<string,
Tensor>(StringComparer.OrdinalIgnoreCase);

            foreach (var outName in transpiler.OutputTensorNames)
            {
                Tensor originalIn = tensors[outName];

                // Создаем НОВЫЙ тензор. Он будет чистым результатом JIT-блока!
                Tensor newOut = new Tensor(originalIn.Shape, context,
MemoryFlags.ReadWrite,
                    requiresGrad: tensors.Values.Any(t => t.RequiresGrad));

                outputDict.Add(outName, newOut);
            }

            // Создаем рабочий словарь для запуска ядра на GPU.
            // Заменяем выходные буферы на новые чистые структуры, чтобы не
затирать входы!
            var executionTensors = new Dictionary<string, Tensor>(tensors,
StringComparer.OrdinalIgnoreCase);
            foreach (var kvp in outputDict)
            {
                executionTensors[kvp.Key] = kvp.Value;
            }
```

```
            //
==========================================================================

            int warp = computeCtx.warp_size;
            int globalX = ((1 + (N / warp)) + computeCtx.MAX_WORK_GPOUP_SIZE_X -
1) / computeCtx.MAX_WORK_GPOUP_SIZE_X * computeCtx.MAX_WORK_GPOUP_SIZE_X;
            int globalY = ((warp) + computeCtx.MAX_WORK_GPOUP_SIZE_Y - 1) /
computeCtx.MAX_WORK_GPOUP_SIZE_Y * computeCtx.MAX_WORK_GPOUP_SIZE_Y;

            Program program = JITProgramCache.GetOrCompile(context,
queue.Device, forwardSource);
            using Kernel kernel =
program.CreateKernel("fused_elementwise_kernel");

            int argIndex = 0;
            foreach (var tensorName in transpiler.DiscoveredTensors)
            {
                Tensor t = executionTensors[tensorName]; // Читает из старых,
пишет в новые!
                kernel.Arguments[argIndex++].SetValue(t.DataBuffer);
                kernel.Arguments[argIndex++].SetValue(t.Size);
            }

            queue.EnqueueNDRangeKernel(kernel, null, new[] { new
IntPtr(globalX), new IntPtr(globalY) }, new[] { new
IntPtr(computeCtx.MAX_WORK_GPOUP_SIZE_X), new
IntPtr(computeCtx.MAX_WORK_GPOUP_SIZE_Y) });
            queue.Finish();

            // Интеграция с Autograd
            bool anyRequiresGrad = tensors.Values.Any(t => t.RequiresGrad);
            // ИСПРАВЛЕНИЕ ИНТЕГРАЦИИ: Убираем дублирование слоев в Autograd

            if (anyRequiresGrad)
            {
                Tensor[] inputs = tensors.Values.ToArray();
                Tensor representative = outputDict.Values.First();

                var localOp = new LocalOp(inputs, representative,
backwardSource, executionTensors, computeCtx);

                // Назначаем Creator СТРОГО только выходным тензорам JIT-блока!
                // Входной projected сохранит свой BmmOp, а Bias останется
листом графа (Creator = null)
                foreach (var outTensor in outputDict.Values)
                {
                    outTensor.Creator = localOp;
                }
            }


            return outputDict; // Возвращаем коллекцию новых результатов
        }


    }
}
```

```
using System;
using System.Collections.Generic;
using NOpenCL;

namespace cstorchcl.core
{
    /// <summary>
    /// Динамический узел графа автоматического дифференцирования для JIT-
сплавленных ядер LocalRunner.
    /// </summary>
    public class LocalOp : Op
    {
        private readonly string _backwardKernelSource;
        private readonly Dictionary<string, Tensor> _tensorArgs;

        // Добавьте поле в класс LocalOp:
        private readonly ComputeContext _computeCtx;

        // Обновите конструктор класса LocalOp, чтобы он принимал
ComputeContext:
        public LocalOp(Tensor[] inputs, Tensor output, string backwardSource,
Dictionary<string, Tensor> tensorArgs, ComputeContext computeCtx)
        {
            Inputs = inputs;
            Output = output;
            _backwardKernelSource = backwardSource;
            _tensorArgs = tensorArgs;
            _computeCtx = computeCtx;
        }

        public override Tensor[] Backward(Tensor gradOutput, Context context)
        {
            CommandQueue queue = DeviceQueueRegistry.GetCurrent();
#if DEBUG
            Console.WriteLine("[locRunner]запуск обратного прохода!");
#endif
            var prg = compiler.JITProgramCache.GetOrCompile(context,
queue.Device, _backwardKernelSource);
            using Kernel kernel =
prg.CreateKernel("fused_elementwise_backward_kernel");

            var outputGrads = new Dictionary<string, Tensor>();
            int argIndex = 0;

            foreach (var kvp in _tensorArgs)
            {
                string name = kvp.Key;
                Tensor t = kvp.Value;

                t.InitGradient(context);

                // Выделяем временный буфер для дельты текущего слоя
                Tensor outLayerGrad = new Tensor(t.Shape, context,
MemoryFlags.ReadWrite, requiresGrad: false);
                outputGrads.Add(name, outLayerGrad);

                //
==========================================================================
                // ИСПРАВЛЕНИЕ СЦЕПКИ ГРАДИЕНТОВ:
                // Если этот тензор в executionTensors является выходом
(например, "P"),
```

```
                // то его входящим градиентом для JIT-математики должен быть
gradOutput,
                // который прислал верхний слой (Softmax), а не его собственный
внутренний t.Grad!
                //
================================================================================
                NOpenCL.Buffer incomingGradBuffer = t.Grad.DataBuffer;

                // Проверяем, совпадает ли буфер с буфером репрезентативного
выхода операции
                if (t.DataBuffer == Output.DataBuffer)
                {
                    incomingGradBuffer = gradOutput.DataBuffer; // Подменяем
градиент на пришедший сверху!
                }
                //
================================================================================

                // Передаем аргументы в Си-ядро OpenCL
                kernel.Arguments[argIndex++].SetValue(t.DataBuffer);
                kernel.Arguments[argIndex++].SetValue(incomingGradBuffer); //
Используем правильный входящий поток градиента!
                kernel.Arguments[argIndex++].SetValue(outLayerGrad.DataBuffer);
                kernel.Arguments[argIndex++].SetValue(t.Size);
            }

            // Нахождение максимального размера сетки потоков
            int maxTensorSize = _tensorArgs.Values.Max(t => t.Size);

            int warp = _computeCtx.warp_size;
            //
================================================================================
            // ИСПРАВЛЕНИЕ: Точное выравнивание 2D варп-сетки для обратного
прохода
            //
================================================================================
            //int maxTensorSize = _tensorArgs.Values.Max(t => t.Size);


            int localX = _computeCtx.MAX_WORK_GPOUP_SIZE_X; // 256
            int localY = _computeCtx.MAX_WORK_GPOUP_SIZE_Y; // 4

            // Вычисляем глобальные размеры строго по вашей формуле
            int rawGlobalY = warp;
            int rawGlobalX = 1 + (maxTensorSize / rawGlobalY);

            // Округляем вверх до кратности локальным группам
            int globalX = ((rawGlobalX + localX - 1) / localX) * localX;
            int globalY = ((rawGlobalY + localY - 1) / localY) * localY;

            IntPtr[] globalWorkSize = new[] { new IntPtr(globalX), new
IntPtr(globalY) };
            IntPtr[] localWorkSize = new[] { new IntPtr(localX), new
IntPtr(localY) }; // Передаем X и Y без путаницы!
            //
================================================================================

            queue.EnqueueNDRangeKernel(kernel, null, globalWorkSize,
localWorkSize);
            queue.Finish();

            queue.Finish();
```

```
            // Формируем результирующий массив градиентов для возврата в
AutoGrad
            // ... (после выполнения ядра queue.Finish(); на странице 3)

            // Формируем чистый результирующий массив изолированных градиентов
для возврата в AutoGrad
            Tensor[] finalResultGrads = new Tensor[Inputs.Length];

            for (int i = 0; i < Inputs.Length; i++)
            {
                Tensor inputTensor = Inputs[i];
                Tensor calculatedGradForLayer = null;

                // Находим, под каким строковым именем этот входной тензор шел в
JIT-блоке
                foreach (var kvp in _tensorArgs)
                {
                    if (kvp.Value == inputTensor)
                    {
                        // Извлекаем буфер градиента, который посчитало наше Си-
ядро
                        calculatedGradForLayer = outputGrads[kvp.Key];
                        break;
                    }
                }

                if (calculatedGradForLayer != null)
                {
                    // Важный шаг (как у BmmOp): сбрасываем создателя, чтобы
этот тензор градиента
                    // считался чистыми данными и не участвовал в обходе графа
                    calculatedGradForLayer.Creator = null;
                    finalResultGrads[i] = calculatedGradForLayer;
                }
                else
                {
                    finalResultGrads[i] = null;
                }
            }

            // Возвращаем изолированные градиенты. Движок AutoGrad сам сложит
их!
            return finalResultGrads;
        }

    }

}
```